

Python: The Language Behind AI

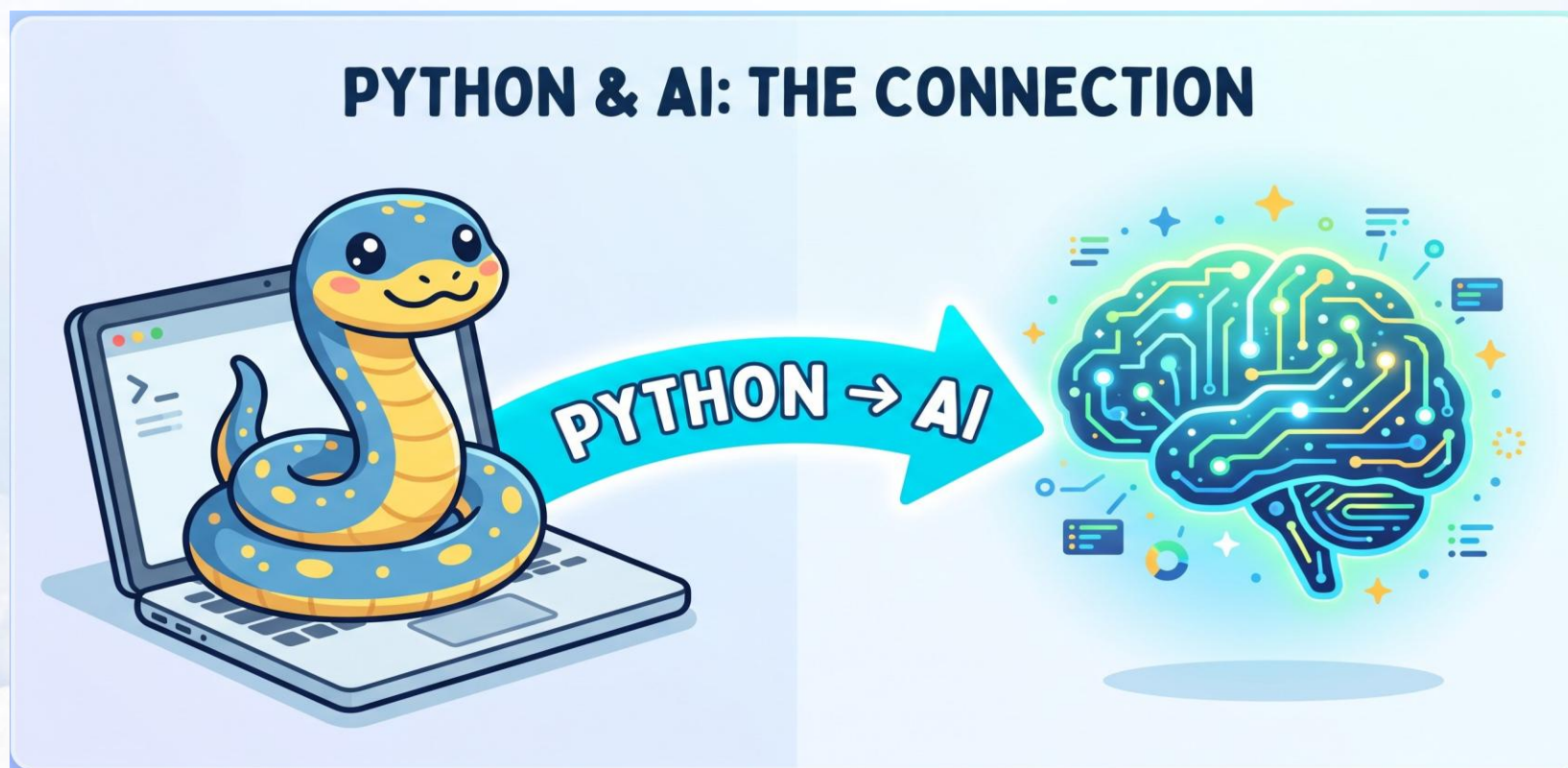
From basics to how AI systems are actually built

Presented by

RA. Muhammad Huzifa

Supervisor

Dr. Muhammad Sajjad



www.huaweicloud.com

Learning Outcomes

CONCEPT EXPLANATION

By the end of this week, students should be able to:

- Explain what a program is and why Python is the interface layer to AI.
- Use variables to hold prompts, scores, and labels.
- Apply the four data types (int, float, str, bool) to AI data.
- Build expressions using operators.
- Write conditionals that model AI decisions.
- Choose the right collection: list, tuple, dict, or set.
- Use loops to repeat operations until a goal is reached.
- Define functions that take input and return output.
- Design a simple class with attributes and methods.

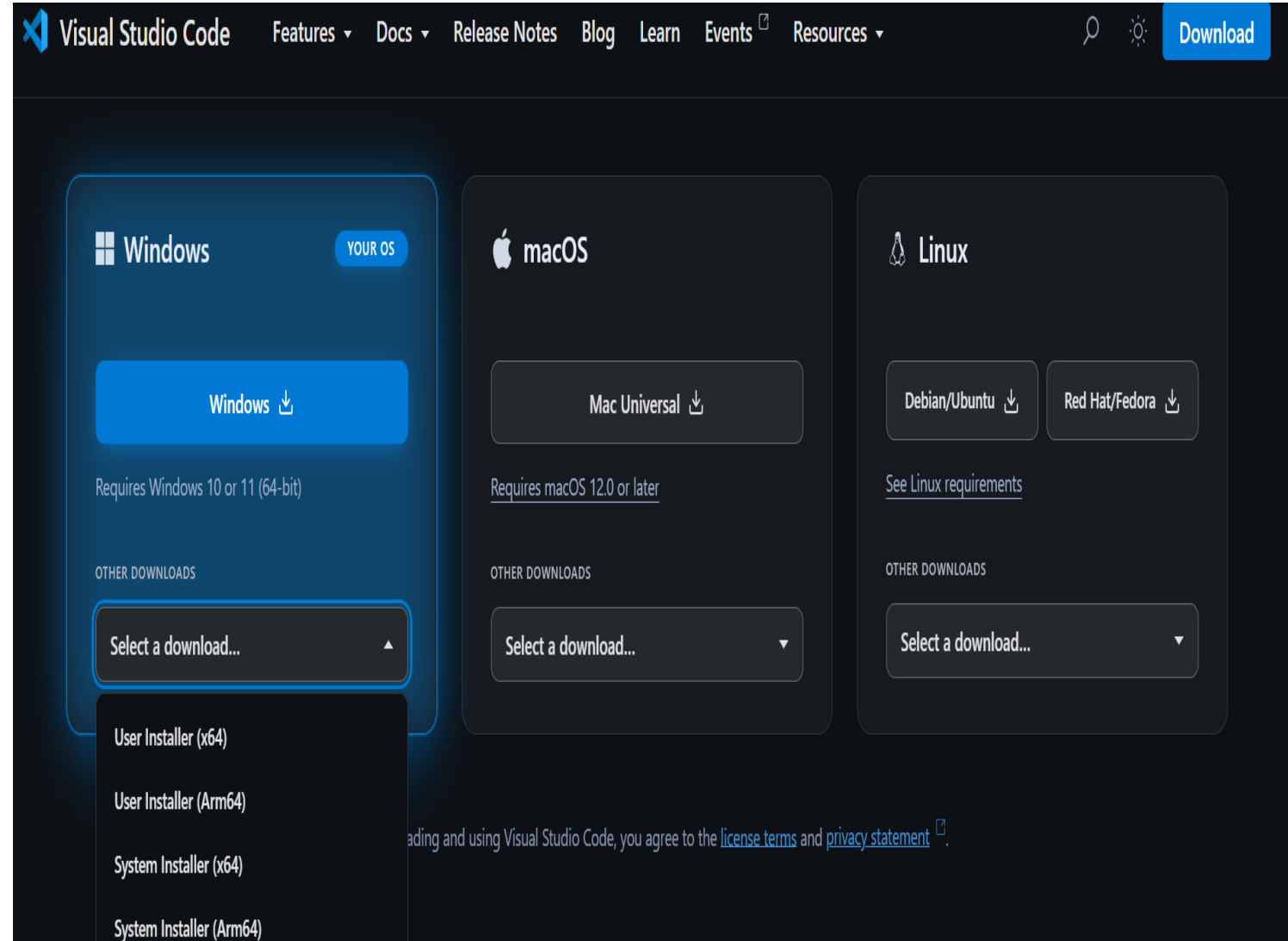
Download & Install python

- Go to the official [Python Downloads Page](#) and click the yellow **Download Python** button.
- Open the downloaded installer file.
- Important:** Before clicking install, check the box at the bottom that says "**Add python.exe to PATH**". *If you skip this, VS Code won't find Python.*
- Click **Install Now** and wait for it to finish.



Download and Install VS CODE

- Head to the official [Visual Studio Code Download Page](#).
- Click the blue **Windows** button (User Installer) to download the setup file.
- Open the downloaded setup file, accept the agreement, and click **Next** through the prompts.
- Click **Install**, then **Finish** to launch VS Code.



Install the python and Jupyter Extension in VS Code

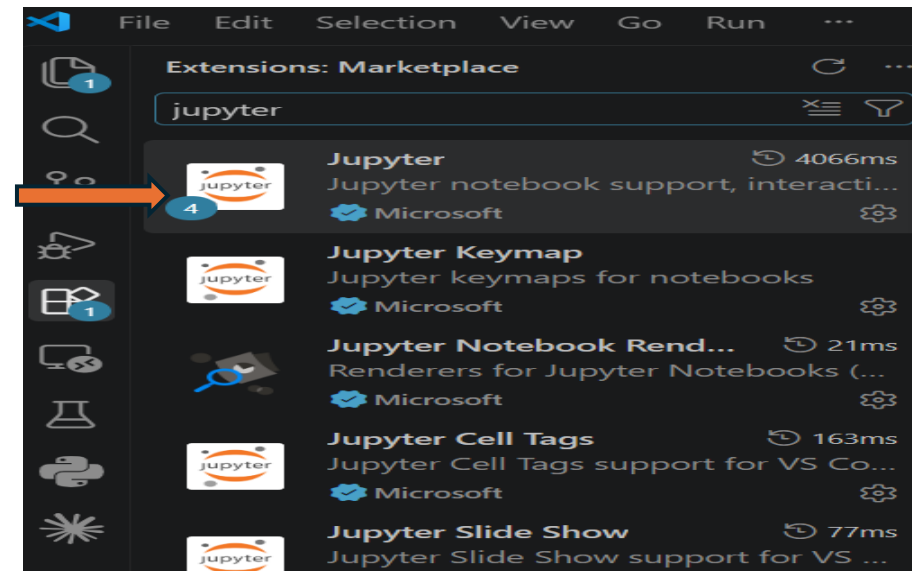
Install the Python Extension in VS Code

1. Open VS Code.
2. Click the **Extensions** icon on the far left sidebar (it looks like 4 blocks, with one flying away).
3. In the search bar at the top, type Python.
4. Find the extension made by **Microsoft** and click the blue **Install** button next to it



Install the Jupyter Extension

1. Open VS Code.
2. Click the **Extensions** icon on the far-left sidebar (the 4 blocks icon).
3. Search for Jupyter in the top search bar.
4. Click **Install** on the main **Jupyter extension** (published by Microsoft)



Create a New Notebook

Create a New Notebook

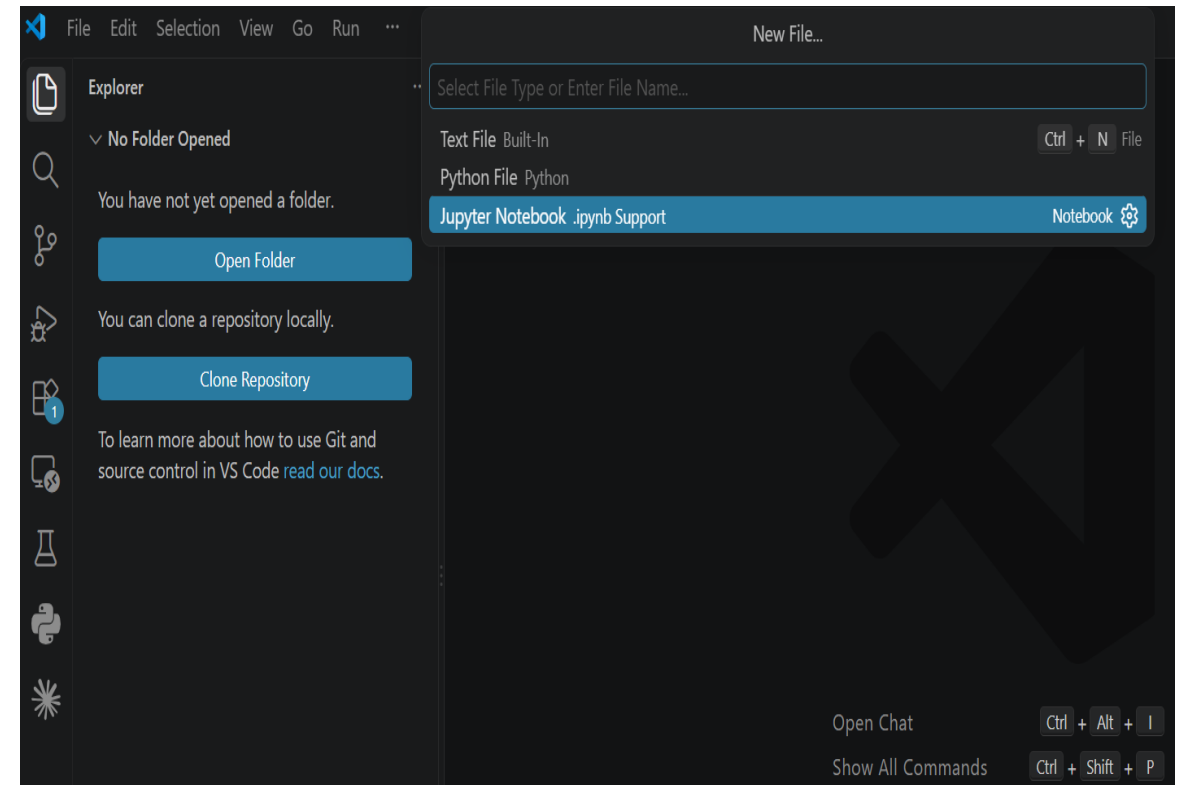
1. Go to the top menu and select **File > New File...**
2. In the dropdown selection that pops up, select **Jupyter Notebook**.

3. Save the file immediately (**File > Save**) and name it something like test.ipynb. Make sure it ends with **.ipynb**.

Step 3: Select Your Python Kernel

Before running code, you must tell the notebook which installation of Python to use:

1. Look at the **top-right corner** of your notebook screen.
2. Click the button that says **Select Kernel** (or click it from the top toolbar).
3. Choose **Python Environments...** and select the standard Python version you installed earlier.

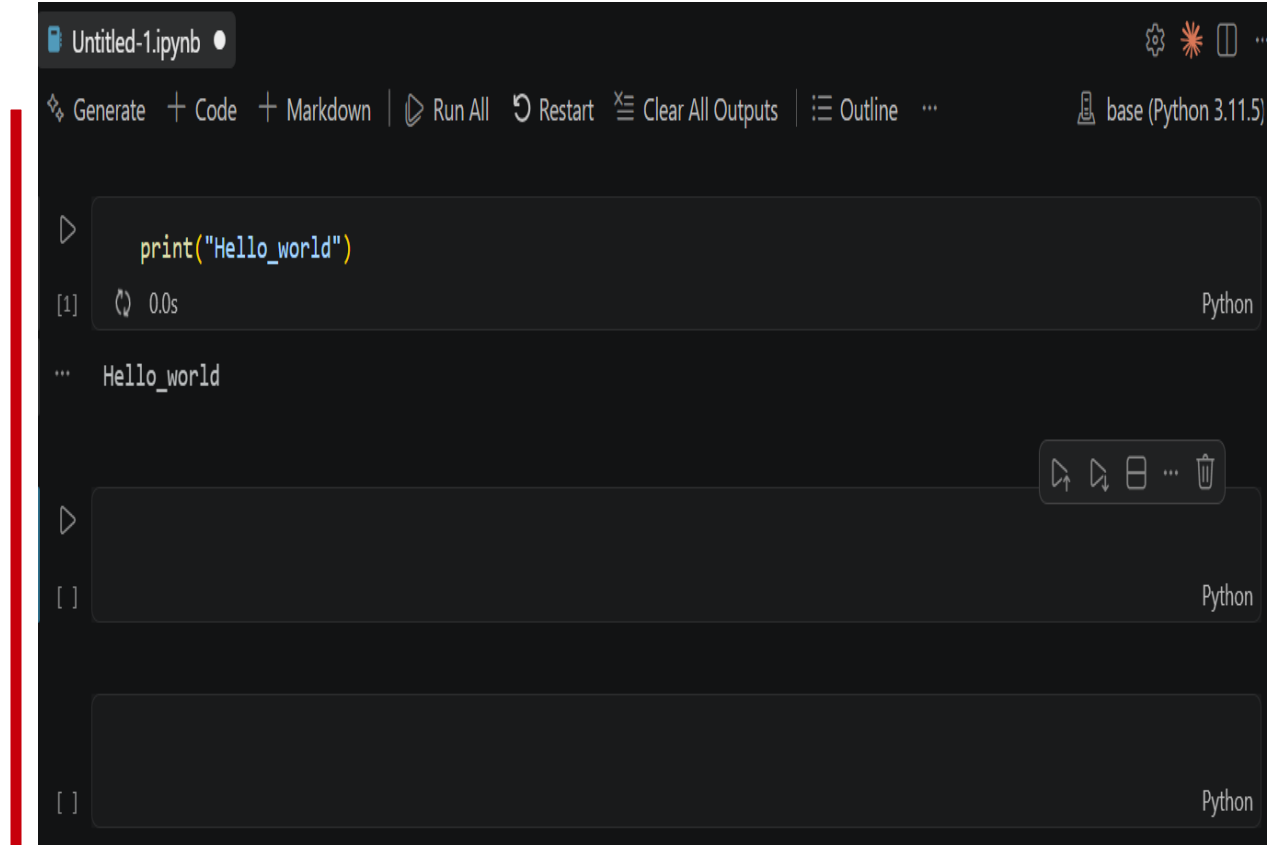


How a Notebook Executes Instructions

A notebook divides a program into small sections called cells.

When a code cell runs, it sends instructions to the Python kernel. The kernel processes those instructions and displays the result below the cell. The notebook remembers variables created during the current session.

Key point: Cells must run in the correct order because later cells may depend on earlier ones. Run it with **shift+enter**

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'Untitled-1.ipynb' and various icons for settings, search, and window management. Below the top bar is a toolbar with buttons for 'Generate', '+ Code', '+ Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Outline', and a menu icon. The main area contains three code cells. The first cell has a play button icon on the left, followed by the code `print("Hello_world")`. Below the code, it shows the execution status '[1] 0.0s' and the output 'Hello_world'. The second and third cells are empty and show '[]' on the left. The bottom right corner of the notebook area has a toolbar with icons for running, undo, redo, and deleting cells. The text 'base (Python 3.11.5)' is visible in the top right corner of the notebook area.

Variable- Holding onto what Matters

What Is a Program?

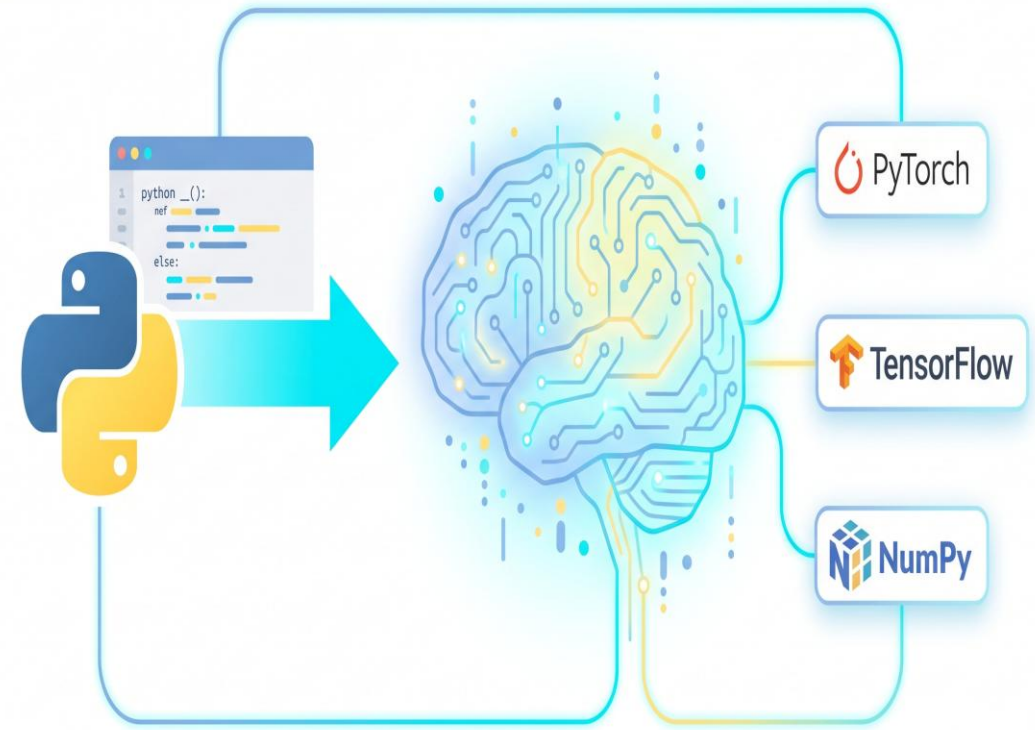
A program is a set of instructions a computer follows, step by step.

- Like a recipe: ingredients (data) + steps (logic) = result
- The computer does exactly what it is told — no guessing, no creativity
- Every AI system — ChatGPT, image generators, recommendation engines — is built from programs

Why Python for AI?

Python is the **interface layer** between you and the AI.

- You write clear instructions in Python
- Python calls powerful libraries (PyTorch, TensorFlow, NumPy) that do the heavy math
- You don't write matrix multiplication by hand — you call a library that does it.



Variable- Holding onto what Matters

What Is a Variable?

A variable is a **name** that points to a **value** stored in memory.

- Think of it as a labeled box: you put something in, give it a name, and use it later
- The name stays the same — the value can change

Why AI Needs Variables

An AI model constantly holds onto things while it

Variable	What It Holds
prompt	The user's input text
score	A confidence value (0 to 1)
label	The predicted class (cat, dog, spam)
weight	A number the model learned during training



Data Types- Numbers, Text, and Truth

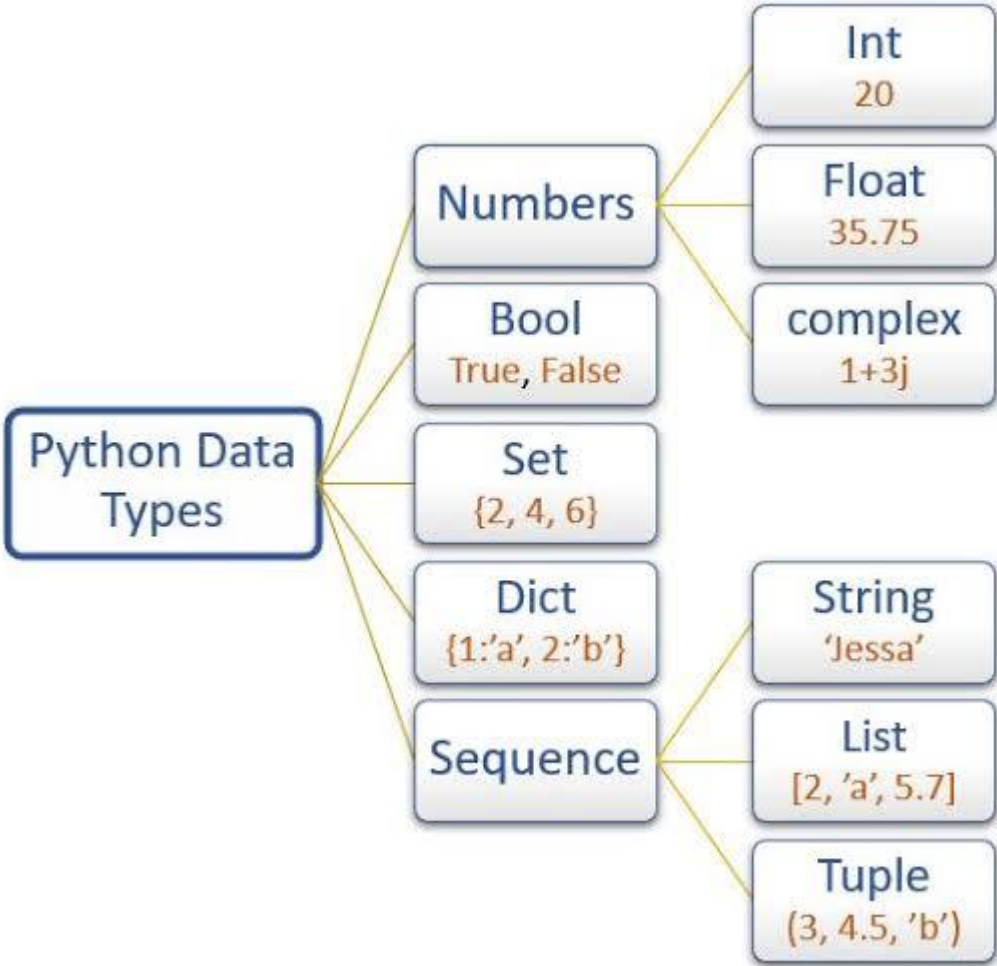
What Is a Data Type?

A data type tells Python what kind of value you are storing — and what you can do with it.

- **Different types = different operations**
- You cannot subtract two words, and you cannot spell-check a number

The Four Core Types

Type	Meaning	Example	Used For
int	Whole number	42	Counts, indexes
float	Decimal number	0.94	Weights, probabilities
str	Text	"hello"	Prompts, tokens, labels
bool	True / False	True	Flags, decisions



Operators & Expressions – Building Calculations

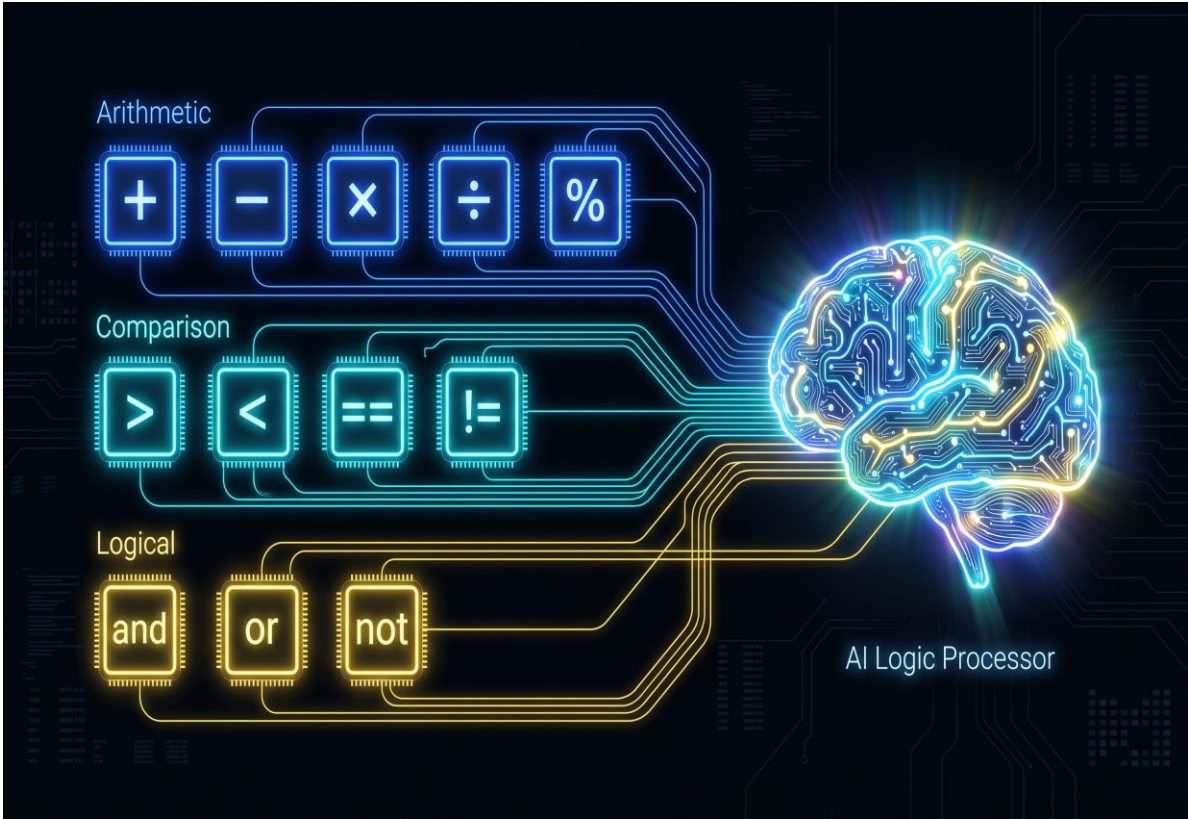
What Is an Operator?

An operator is a symbol that performs an action on values.

- **Operand** = the value being worked on
- **Operator** = the symbol that acts on it
- **Expression** = operands + operators → produces a result

Types of Operators

Category	Operators	Example	Result
Arithmetic	+ - * / // % **	10 + 5	15
Comparison	== != > < >= <=	10 > 5	True
Logical	and or not	True and False	False
Assignment	= += -=	x += 1	increment x



Conditionals — How AI Makes Decisions

What Is a Conditional?

A conditional lets a program choose between different actions based on a condition.

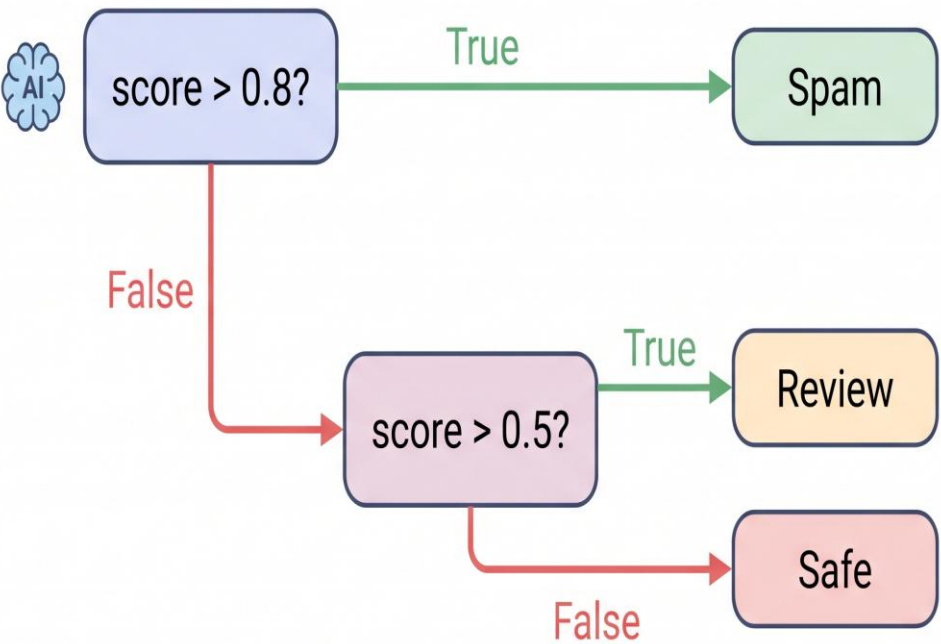
If the condition is True → do one thing

Otherwise → do something else

Every decision an AI makes is a conditional at its core.

Why AI Needs Conditionals

AI Decision	Conditional
Is this email spam?	if score > 0.8:
Accept or reject a response?	if confidence >= 0.9:
Which class has highest score?	if cat > dog and cat > bird:
Should the model send an alert?	if risk > threshold:



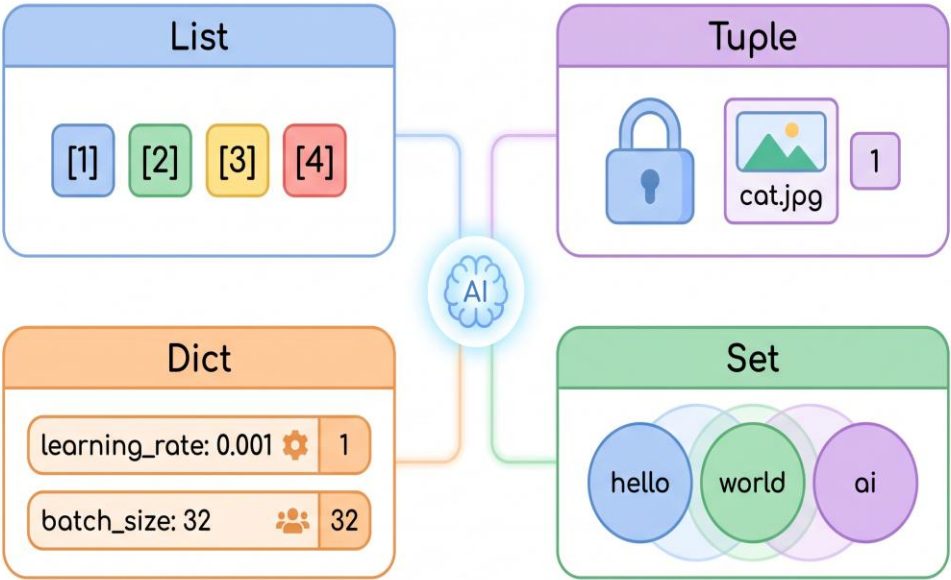
Collections — Organizing Many Values

What Is a Collection?

- A collection is a container that holds multiple values under one name.
- Instead of creating 100 variables, you create one collection
- Each collection type has its own purpose and its own rules
- Choosing the right one makes your code cleaner and faster

The Four Core Collections

Collection	Ordered?	Mutable?	Duplicates	Use For
List []	✓	✓	✓	Datasets, sequences
Tuple ()	✓	✗	✓	Fixed pairs (feature, label)
Dict { key: value }	✓	✓	keys unique	Configs, lookups
Set { }	✗	✓	✗	Unique vocabulary, tags



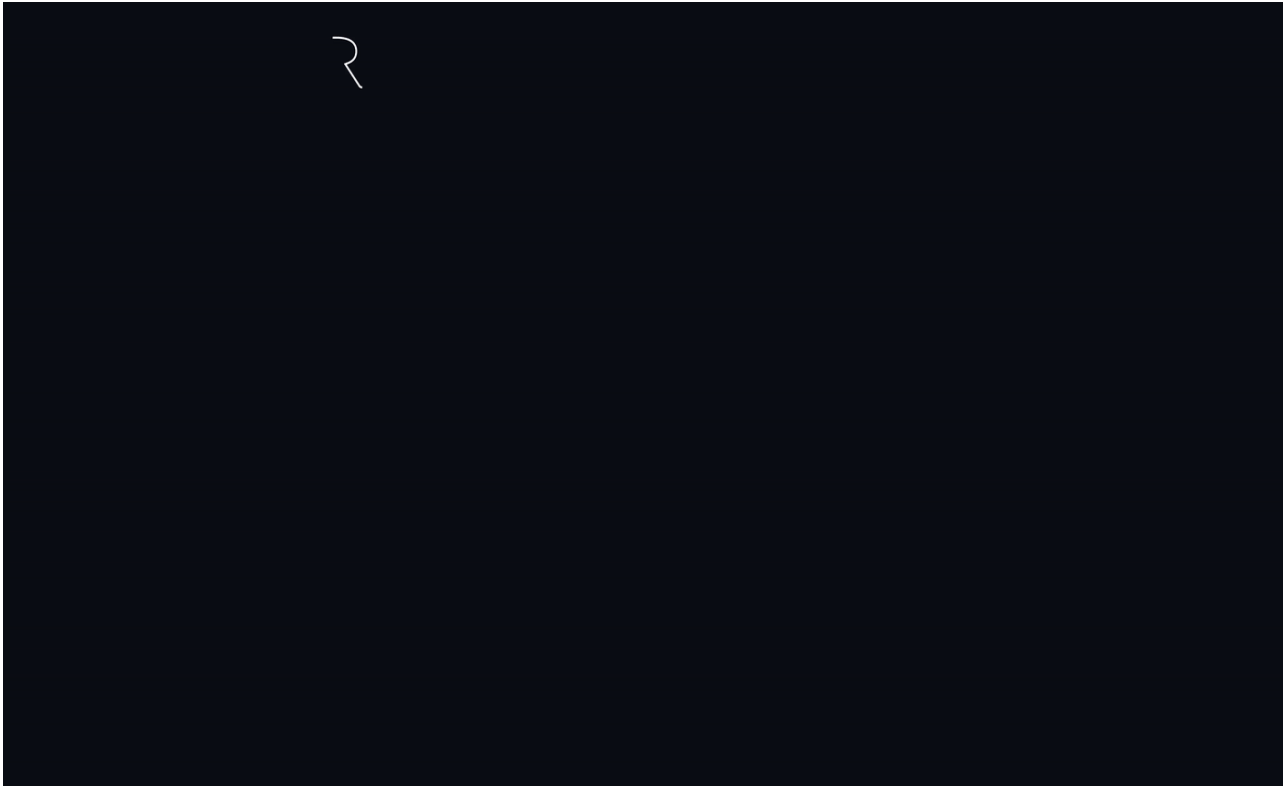
Loop — Repeat Until Done

What Is a for Loop?

- A for loop repeats a block of code once for each item in a collection.
- You write the instruction once
- Python runs it for every item automatically
- The loop stops when there are no more items

Why AI Needs for Loops

AI Task	What the for Loop Does
Load every image in a dataset	Visits each image one by one
Process every token in a sentence	Visits each token
Train for 100 epochs	Repeats the training step 100 times
Evaluate every test sample	Predicts and compares one by one



What Is a Nested Loop?

A nested loop is a loop inside another loop.
The outer loop runs once
The inner loop runs completely for each outer iteration
Total iterations = outer × inner
The animations of 3x3 matrix is in next slide.

3x3 Matrix Multiplication

$\wedge$

Functions — Reusable Logic

What Is a Function?

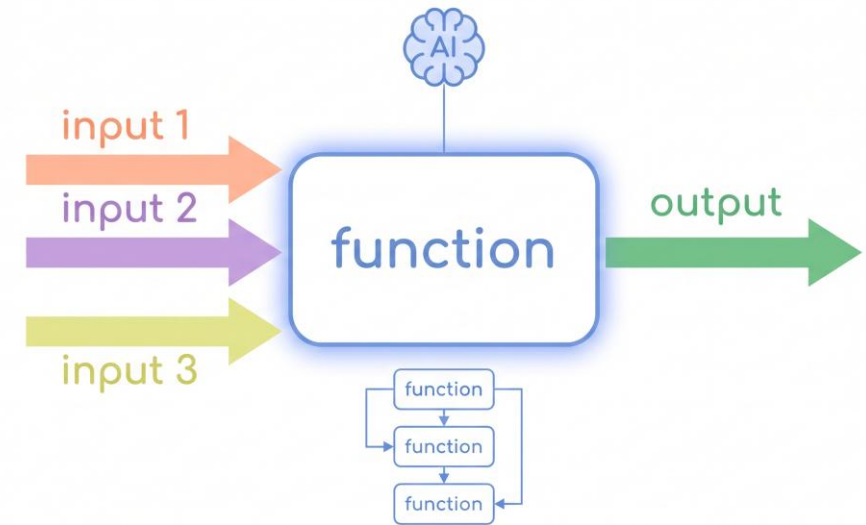
- A function is a named block of code that performs one task.
- Write it once, use it many times
- It takes input, processes it, and returns output
- If you need the same logic again, you just call the function by name

A function is the basic form of what we later call a model or a tool — input in, output out.

Why AI Needs Functions

Every model you will ever use is a function at its core:

AI Concept	What the Function Does
A classifier	Takes input features → returns a prediction
A tokenizer	Takes text → returns a list of tokens
A loss function	Takes predictions + labels → returns a score
An activation function	Takes a number → returns a transformed number
A model's forward pass	Takes input → returns output



Intro to OOP — Bundling State and Behavior

What Is a Class?

- A class is a blueprint that bundles two things together:
- Attributes — the data (state, memory)
- Methods — the functions (behavior, actions)
- An object is a specific instance created from the class blueprint.

If a function is a single tool, a class is a toolbox — the tools plus the drawer that holds them.

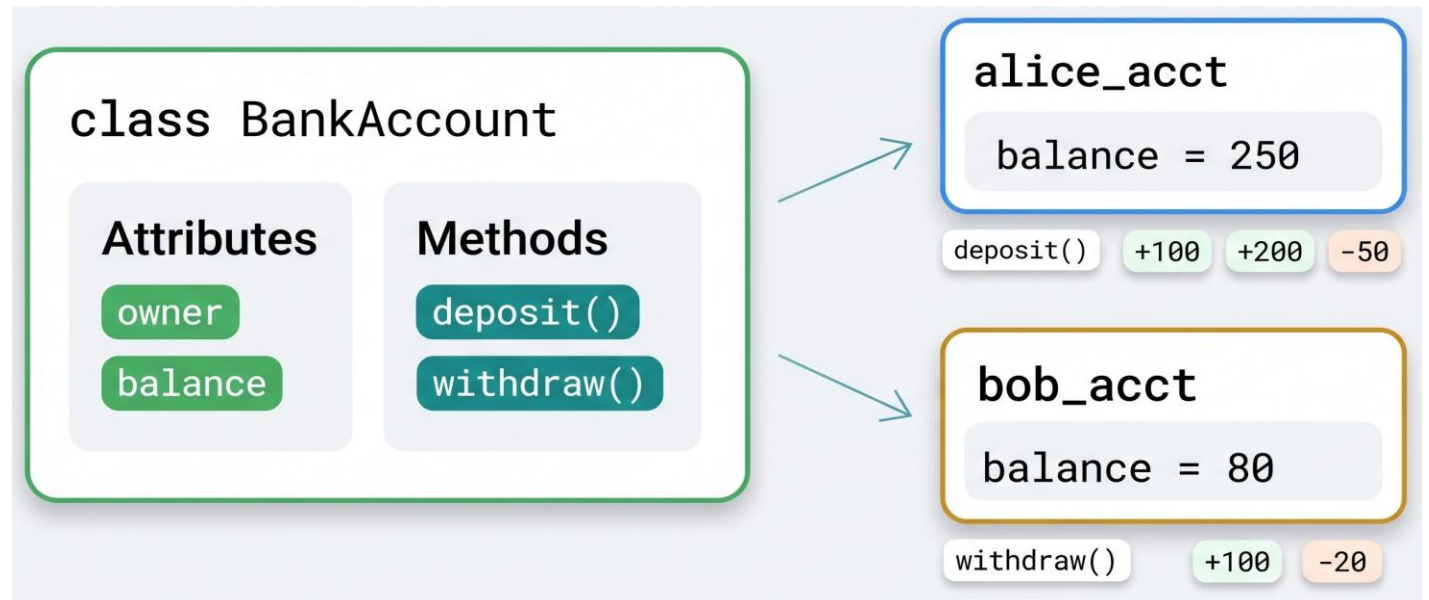
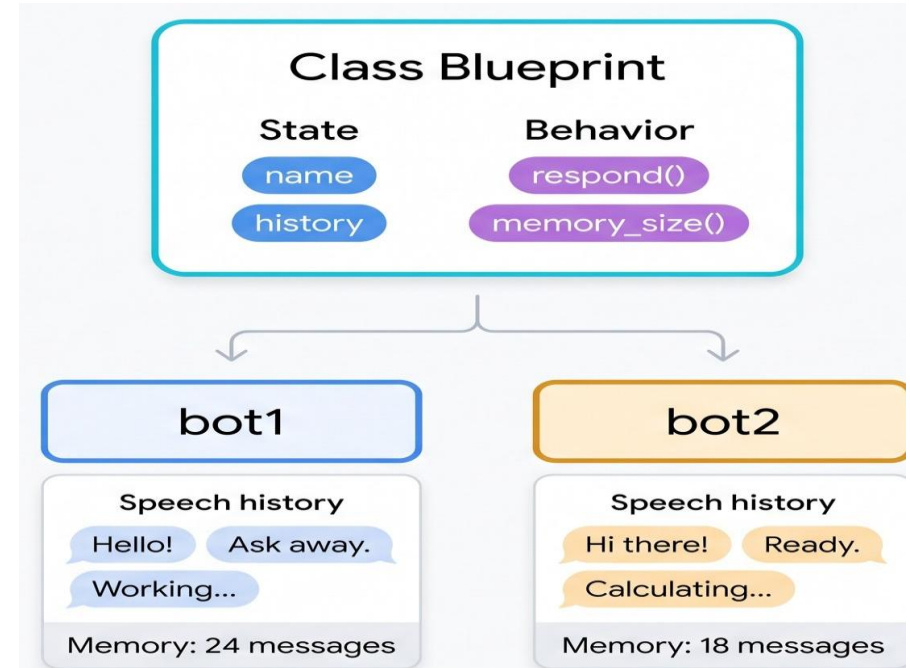
class — tells Python you are defining a class

__init__ — runs when you create a new object (sets initial state)

self — refers to the specific object being used

Attributes store data

Methods act on that data



One Program, Many Concepts

A real program combines every concept you have learned:

Concept	Role in the Program
Variables	Hold the data (prompts, scores, labels)
Data types	Classify the data (int, float, str, bool)
Operators	Act on the data (arithmetic, comparison, logic)
Conditionals	Decide what to do (if / elif / else)
Collections	Organize the data (list, tuple, dict, set)
Loops	Repeat the work (for / while)
Functions	Package logic into reusable pieces
Classes	Bundle memory and behavior into one object



Thank You

Any Question?